

# Graph Optimization Guide

---

## Overview

---

The graph optimization module provides tools for simplifying, fusing, and serializing BDI computation graphs. These optimizations reduce memory usage, improve cache locality, and enable better parallelization.

## Core Concepts

---

### Dead Node Elimination

Dead nodes are nodes that:

1. Have no users (no other nodes depend on them)
2. Are not output nodes (OP\_RET)
3. Have no side effects

**Algorithm:** Backward liveness propagation

1. Mark all output nodes as live
2. Propagate liveness backward through dependencies
3. Remove all unmarked nodes

### Constant Folding

Merges duplicate constant values to reduce memory footprint.

**Benefits:**

- Reduced memory usage
- Improved cache locality
- Simplified dependency tracking

### Subgraph Fusion

Identifies chains of operations that can be fused into single kernels.

**Fusion Candidates:**

- Arithmetic chains (ADD→MUL→ADD)
- Element-wise operations
- Reduction operations

**Fusion Score:** Heuristic based on:

- Chain length
- Operation types
- Memory access patterns

## API Reference

---

### Graph Simplification

```
int graph_simplify(BdiGraph* graph);
```

Performs all optimization passes on the graph.

**Returns:** 0 on success, -1 on error

**Example:**

```
BdiGraph* graph = /* ... */;  
if (graph_simplify(graph) == 0) {  
    printf("Graph optimized successfully\n");  
}
```

### Dead Node Removal

```
int graph_remove_dead_nodes(BdiGraph* graph);
```

Removes unreachable nodes from the graph.

**Returns:** Number of nodes removed, or -1 on error

### Constant Merging

```
int graph_merge_constants(BdiGraph* graph);
```

Merges duplicate constant values.

**Returns:** Number of constants merged, or -1 on error

### Subgraph Identification

```
Subgraph* identify_fusible_subgraph(  
    const BdiGraph* graph,  
    size_t* out_count  
);
```

Identifies fusible subgraphs in the computation graph.

**Parameters:**

- graph : Input graph
- out\_count : Output parameter for number of subgraphs found

**Returns:** Array of subgraphs, or NULL on error

**Example:**

```

size_t count = 0;
Subgraph* subgraphs = identify_fusible_subgraph(graph, &count);

for (size_t i = 0; i < count; i++) {
    printf("Subgraph %zu: %zu nodes, score=%.2f\n",
        i, subgraphs[i].count, subgraphs[i].fusion_score);
    subgraph_free(&subgraphs[i]);
}
free(subgraphs);

```

## Subgraph Fusion

```

int fuse_subgraph(BdiGraph* graph, const Subgraph* subgraph);

```

Fuses a subgraph into a single node.

**Returns:** 0 on success, -1 on error

## Serialization Format

### Binary Format Specification

```

GraphHeader (32 bytes)
┌─ magic: 0x42444947 ("BDIG")
┌─ version: 1
┌─ node_count: uint64_t
┌─ edge_count: uint64_t
└─ checksum: uint64_t

GraphNode[] (variable)
┌─ node[0]
┌─ node[1]
└─ ...

GraphEdge[] (variable)
┌─ edge[0]
┌─ edge[1]
└─ ...

```

## Serialization API

```

int graph_serialize(const BdiGraph* graph, const char* path);

```

Saves graph to binary file.

**Example:**

```

if (graph_serialize(graph, "model.bdig") == 0) {
    printf("Graph saved successfully\n");
}

```

## Deserialization API

```
BdiGraph* graph_deserialize(const char* path);
```

Loads graph from binary file.

### Example:

```
BdiGraph* graph = graph_deserialize("model.bdig");  
if (graph) {  
    printf("Loaded %zu nodes\n", graph->node_count);  
}
```

## Optimization Strategies

### When to Optimize

#### Before Execution:

- Remove dead code
- Merge constants
- Identify fusion opportunities

#### During Execution:

- Profile hot paths
- Adaptive fusion based on runtime data

#### After Execution:

- Serialize optimized graphs for reuse

### Optimization Pipeline

```
Input Graph  
  ↓  
Dead Node Removal  
  ↓  
Constant Folding  
  ↓  
Subgraph Identification  
  ↓  
Selective Fusion  
  ↓  
Optimized Graph
```

## Performance Considerations

### Time Complexity

| Operation         | Complexity     | Notes                                        |
|-------------------|----------------|----------------------------------------------|
| Dead Node Removal | $O(n^2)$       | Can be optimized with better data structures |
| Constant Merging  | $O(n^2)$       | Pairwise comparison                          |
| Subgraph ID       | $O(n \cdot m)$ | $m$ = average inputs per node                |
| Serialization     | $O(n)$         | Linear write                                 |
| Deserialization   | $O(n)$         | Linear read + validation                     |

### Space Complexity

| Operation         | Space          | Notes                         |
|-------------------|----------------|-------------------------------|
| Liveness Analysis | $O(n)$         | Boolean array                 |
| Subgraph Storage  | $O(k \cdot m)$ | $k$ subgraphs, $m$ nodes each |
| Serialization     | $O(n)$         | Output buffer                 |

## Best Practices

### 1. Optimize Early

```
// Good: Optimize before execution
graph_simplify(graph);
execute_graph(graph);

// Bad: Execute unoptimized graph
execute_graph(graph);
```

### 2. Cache Optimized Graphs

```
// Save optimized graph
graph_serialize(optimized_graph, "cache/model_opt.bdig");

// Reuse in future runs
BdiGraph* graph = graph_deserialize("cache/model_opt.bdig");
```

### 3. Profile Before Fusion

```
// Identify hot subgraphs
size_t count = 0;
Subgraph* subgraphs = identify_fusible_subgraph(graph, &count);

// Fuse only high-score subgraphs
for (size_t i = 0; i < count; i++) {
    if (subgraphs[i].fusion_score > 2.0f) {
        fuse_subgraph(graph, &subgraphs[i]);
    }
}
```

## Debugging

### Validation

```
// Verify graph integrity after optimization
bool validate_graph(const BdiGraph* graph) {
    // Check node IDs are unique
    // Verify edge connectivity
    // Validate type consistency
    return true;
}
```

### Visualization

```
// Export graph to DOT format for visualization
void export_to_dot(const BdiGraph* graph, const char* path) {
    FILE* f = fopen(path, "w");
    fprintf(f, "digraph G {\n");

    for (size_t i = 0; i < graph->node_count; i++) {
        fprintf(f, "    n%llu [label=\"%s\"]; \n",
            graph->nodes[i].id,
            opcode_to_string(graph->nodes[i].opcode));
    }

    // ... edges ...

    fprintf(f, "}\n");
    fclose(f);
}
```

## Troubleshooting

### Common Issues

**Issue:** Serialization fails with checksum mismatch

**Solution:** Ensure graph is not modified during serialization

**Issue:** Fusion creates invalid dependencies

**Solution:** Verify all external inputs are captured

**Issue:** Dead node removal is too aggressive

**Solution:** Mark side-effect nodes as live

## Examples

---

### Complete Optimization Pipeline

```
#include "kernel/graph_opt/graph_opt.h"

int optimize_and_save(BdiGraph* graph, const char* output_path) {
    // Step 1: Remove dead nodes
    int removed = graph_remove_dead_nodes(graph);
    printf("Removed %d dead nodes\n", removed);

    // Step 2: Merge constants
    int merged = graph_merge_constants(graph);
    printf("Merged %d constants\n", merged);

    // Step 3: Identify fusion opportunities
    size_t count = 0;
    Subgraph* subgraphs = identify_fusible_subgraph(graph, &count);
    printf("Found %zu fusible subgraphs\n", count);

    // Step 4: Fuse high-value subgraphs
    for (size_t i = 0; i < count; i++) {
        if (subgraphs[i].fusion_score > 1.5f) {
            fuse_subgraph(graph, &subgraphs[i]);
        }
        subgraph_free(&subgraphs[i]);
    }
    free(subgraphs);

    // Step 5: Serialize optimized graph
    return graph_serialize(graph, output_path);
}
```

## See Also

---

- Device Backend Guide: Execution of optimized graphs
- Scheduler Guide: Parallel execution strategies
- HAM Intelligence Guide: Memory optimization